BURP SUITE: REPEATER
TryHackMe Learning & Practical Report

Prepared by: Kaustubh Ashish Bangar

Focus: Web Application Security / WAPT Foundations

Task: Task 3 — Burp Suite

Room Covered: Burp Suite: Repeater

A comprehensive technical report documenting the methodologies, theoretical concepts, and practical exercises completed for the Burp Suite framework phase of the IqraSec Academy WAPT Internship Phase 1.

1. Introduction

In professional Web Application Penetration Testing (WAPT), automated vulnerability scanners frequently fail to identify complex business logic flaws, authorization bypasses, and nuanced injection vulnerabilities. To uncover these critical security issues, an analyst must manually intercept, inspect, and manipulate raw HTTP/HTTPS traffic. Burp Suite, developed by PortSwigger, is the industry-standard framework for this exact purpose.

While the Burp Proxy module is utilized to intercept traffic in real-time, the Burp Suite Repeater module serves as the primary workbench for deep, manual vulnerability exploitation. Repeater allows security analysts to capture a specific HTTP request, isolate it from the browser, manually modify its parameters, headers, or body, and repeatedly transmit it to the target server to observe the corresponding responses.

This technical report comprehensively documents the architecture, interface functionalities, and practical application of the Burp Suite Repeater module. It details manual header manipulation, unhandled exception triggering, and the step-by-step exploitation of a Union-based SQL Injection vulnerability.

2. Objective

The primary objective of this module is to master the manual manipulation of HTTP requests using Burp Suite Repeater to identify and exploit web application vulnerabilities.

Specific learning and practical objectives include:

1. **Framework Navigation:** To understand the graphical interface, layout options, and core functionalities of the Burp Suite Repeater module.
2. **Request Manipulation:** To capture intercepted requests from the Proxy module and forward them to Repeater for iterative testing.
3. **Inspector Utilization:** To utilize the Inspector tool for the intuitive, tabular breakdown and modification of request attributes, headers, and parameters.
4. **Response Analysis:** To analyze server responses using various rendering views (Pretty, Raw, Hex, Render) and identify hidden non-printable characters.

5. **Access Control Bypass:** To manually inject custom HTTP headers into a request to bypass simulated access controls.
6. **Input Validation Testing:** To tamper with URI parameters using extreme inputs to test application error handling and trigger `500 Internal Server Error` exceptions.
7. **Vulnerability Exploitation (SQLi):** To independently identify, enumerate, and exploit a Union-based SQL Injection vulnerability by meticulously crafting payloads within Repeater.

3. Concept / Understanding

3.1 The Purpose of Burp Suite Repeater

In essence, Burp Suite Repeater enables penetration testers to modify and resend intercepted requests to a chosen target. It completely removes the need to constantly refresh a web browser and navigate through a web application's GUI to test a single input field. By capturing a request and sending it to Repeater (via the `Ctrl + R` shortcut), an analyst can manually create or alter requests from scratch, functioning similarly to a command-line tool like cURL, but with a highly advanced, user-friendly graphical interface. The ability to edit and resend requests repeatedly makes Repeater an invaluable asset for the manual exploration and exploitation of API endpoints and web forms.

3.2 Repeater Interface Architecture

The Repeater interface is systematically divided into six primary sections to streamline the testing workflow:

1. **Request List:** Located at the top left, this tabbed interface displays the active list of Repeater requests. Analysts can manage multiple, distinct requests simultaneously, allowing for parallel testing of different endpoints.
2. **Request Controls:** Positioned directly beneath the request list, these controls include the primary "Send" button to dispatch the request, a "Cancel" button for hanging or timed-out requests, and history navigation arrows to seamlessly move forward or backward through previous modifications.

3. **Request and Response View:** Occupying the majority of the interface, this dual-pane section allows the analyst to directly edit the raw HTTP request on the left and observe the corresponding server HTTP response on the right.
4. **Layout Options:** Located at the top-right, these toggles allow the user to customize the workspace layout, offering side-by-side (horizontal) views, over-under (vertical) views, or completely separate tabs based on monitor constraints and payload size.
5. **Target:** Situated above the Inspector on the right, this field explicitly specifies the destination IP address, hostname, port, and protocol (HTTP/HTTPS). It is automatically populated when sending a request from the Proxy.
6. **Inspector:** Positioned on the far-right side, the Inspector tool provides a visually organized, tabular breakdown of the request and response components, allowing for intuitive modifications without breaking raw HTTP syntax.

3.3 Response Presentation Options

Repeater provides multiple presentation options to analyze the server's response:

- **Pretty:** The default option, which takes the raw response and applies slight formatting enhancements, such as syntax highlighting and indentation, to significantly improve readability for HTML, JSON, or XML.
- **Raw:** Displays the exact, unmodified response directly received from the server, which is crucial for identifying malformed responses or hidden padding.
- **Hex:** Examines the response in a byte-level hexadecimal representation. This is specifically required when dealing with binary files, serialized data, or identifying invisible byte-order marks.
- **Render:** Utilizes a built-in rendering engine to visualize the HTML response exactly as it would graphically appear in a standard web browser.
- **Show Non-Printable Characters (`\n`):** Enables the display of invisible characters, such as Carriage Return (`\r`) and Line Feed (`\n`). Because HTTP headers rely strictly on `\r\n` line endings to function, this feature is critical for debugging HTTP Request Smuggling attacks.

3.4 The Inspector Module

The Inspector is a supplementary feature that breaks down complex HTTP messages into easily editable components. This prevents analysts from accidentally introducing syntax errors (like missing a space or a colon) when manually typing in the raw view.

- **Request Attributes:** Allows the modification of the HTTP method (e.g., swapping `GET` to `POST` to test for method-based routing flaws), the requested URI path, and the HTTP protocol version.
- **Request Query Parameters:** Isolates data sent to the server via the URL query string (e.g., `?id=1`), allowing for rapid tampering.
- **Request Body Parameters:** Specific to `POST` or `PUT` requests, isolating form data or JSON payloads submitted in the message body.
- **Request Headers:** Provides a modifiable list of all headers. Editing these is highly valuable when examining how a web server processes unexpected headers (e.g., `X-Forwarded-For` for IP spoofing).
- **Request Cookies:** Isolates session tokens, making it easy to paste in hijacked cookies or test for session fixation vulnerabilities.

4. Practical Work

The practical application phase of this module required intercepting standard web traffic and utilizing Burp Suite Repeater to systematically break application logic, bypass controls, and extract sensitive database records.

4.1 Header Manipulation (Bypassing Controls)

The first practical exercise demonstrated how easily client-side requests can be manipulated to bypass server-side checks.

- **Execution:** I intercepted a standard `GET / HTTP/1.1` request destined for the target server and forwarded it to Repeater.
- **Manipulation:** Utilizing the raw Request view, I injected a custom, non-standard HTTP header mandated by the challenge: `FlagAuthorised: True`.

- **Result:** Upon sending the modified request, the backend application logic processed the custom header, verified the authorization bypass, and returned an HTTP `200 OK` response containing the first flag (`THM{Yzg2MWI2ZDhlYzdINGFiZTU2ZTIzMzVi}`) within the response body.

4.2 Endpoint Validation and Error Triggering

The second exercise focused on input validation testing, specifically targeting how an application handles unexpected or extreme parameter inputs.

- **Execution:** I navigated the application to a numeric product endpoint (e.g., `/products/3`), intercepted the request via Burp Proxy, and forwarded it to Repeater.
- **Manipulation:** The objective was to validate if the endpoint strictly enforced valid integer inputs. In the Request view, I modified the URI path parameter from a valid product ID (`/products/3`) to an extreme, negative input (`/products/-1`).
- **Result:** The application failed to properly sanitize or validate the negative integer, causing an unhandled exception in the backend code. The server explicitly crashed, returning an HTTP `500 Internal Server Error` alongside the second flag (`THM{N2MzMzFhMTA1MmZiYjA2YWQ4M2ZmMzhl}`). This demonstrated that the endpoint was fundamentally unstable and potentially vulnerable to further injection attacks.

4.3 Extra-Mile Challenge: Union SQL Injection Exploitation

The final, most advanced challenge required leveraging Burp Suite Repeater to perform a manual Union-based SQL Injection attack against the `/about/ID` endpoint to extract hidden CEO notes from the database.

- **Step 1: Vulnerability Identification** I intercepted a request to `/about/2` and sent it to Repeater. To test for SQLi, I manually appended a single apostrophe (`'`) to the ID parameter in the URI (`GET /about/2' HTTP/1.1`).
- **Step 2: Error Analysis** The server responded with a `500 Internal Server Error`. Crucially, the response body contained an overly verbose error message that leaked the exact backend SQL query: `SELECT firstName, lastName, pfpLink, role, bio FROM people WHERE id = 2'`. This devastating information disclosure revealed the target table (`people`) and confirmed the query expected exactly five columns.

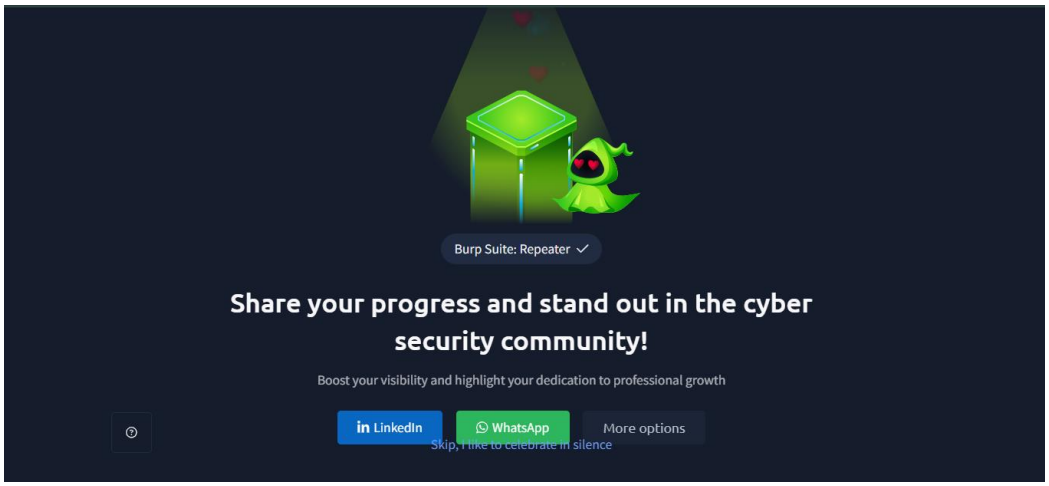
- **Step 3: Column Enumeration (Information Schema)** Knowing the column count, I utilized a `UNION ALL SELECT` statement to query the database's structural metadata. I changed the target ID to `0` (an invalid user, ensuring my injected query results populated the first row) and injected the following payload: `/about/0 UNION ALL SELECT column_name,null,null,null,null FROM information_schema.columns WHERE table_name="people"` The response title successfully reflected the first column name (`id`), confirming the Union query executed perfectly.
- **Step 4: Grouping Results** Because the application only displayed a single result at a time, I utilized the `group_concat()` SQL function to amalgamate all column names into a single output string. `/about/0 UNION ALL SELECT group_concat(column_name),null,null,null,null FROM information_schema.columns WHERE table_name="people"` This revealed eight columns: `id`, `firstName`, `lastName`, `pfpLink`, `role`, `shortRole`, `bio`, and `notes`.
- **Step 5: Data Extraction** With the table (`people`), the target column (`notes`), and the CEO's ID (`1`) identified, I crafted the final exploit payload in Repeater: `GET /about/0 UNION ALL SELECT notes,null,null,null,null FROM people WHERE id = 1 HTTP/1.1`
- **Result:** The server processed the injected query, bypassed the standard profile retrieval, and successfully returned the highly sensitive notes column containing the final flag: `THM{ZGE3OTUyZGMyMzkwNjJmZjg3Mzk1NjJh}`.

5. Tools Used

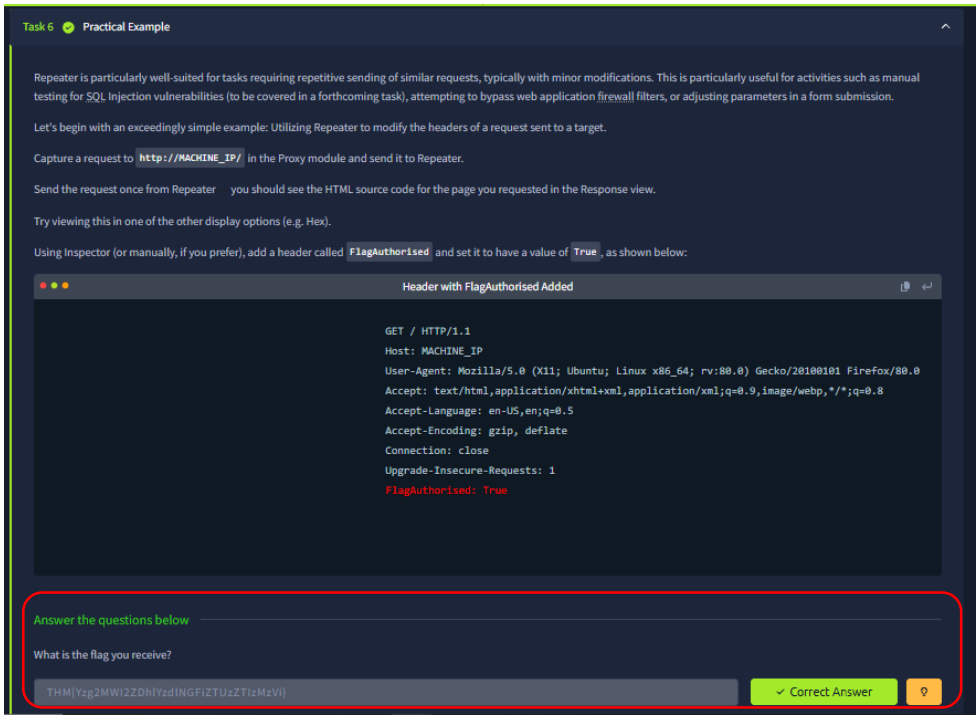
- **Burp Suite Community Edition (v2026.2.3):** The primary intercepting proxy framework utilized for the assessment.
- **Burp Proxy:** Used to capture the initial HTTP requests from the browser before routing them to Repeater.
- **Burp Repeater:** The core module utilized for all manual payload crafting, header injection, and iterative response analysis.
- **Repeater Inspector:** Utilized to intuitively review request attributes and header parameters.
- **TryHackMe AttackBox:** The browser-based virtual environment utilized to execute the attacks.

6. Evidence and Screenshots

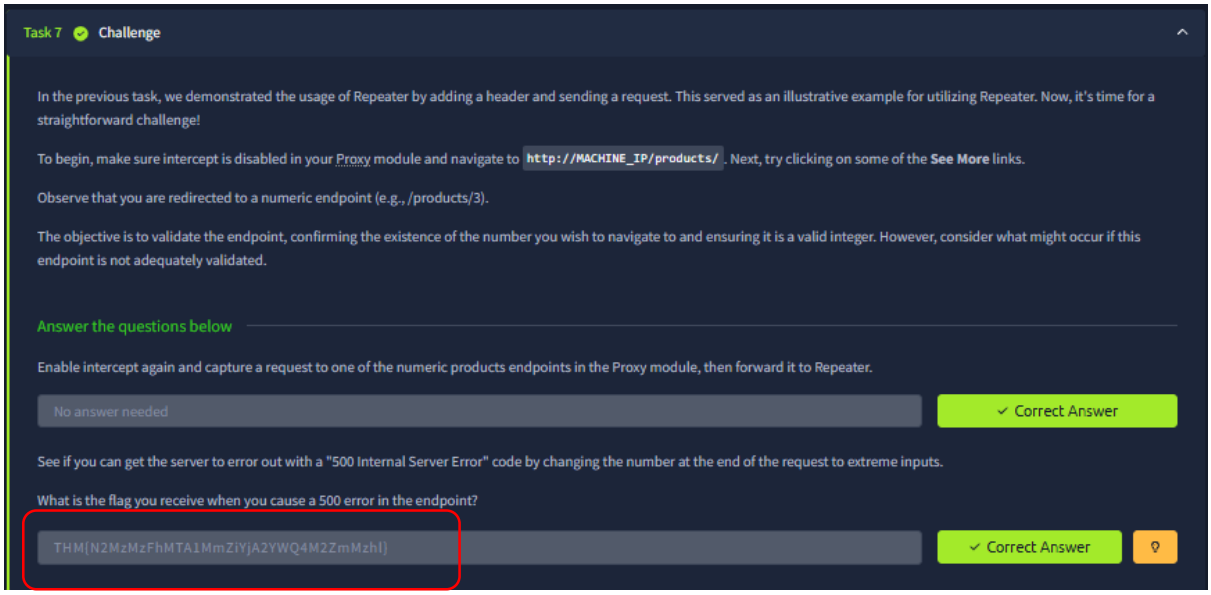
All relevant visual evidence documenting the successful completion of these exercises is stored in the designated `Kaustubh_Bangar_WAPT_Phase-1/3.Burp-Suite/` *submission directory.*



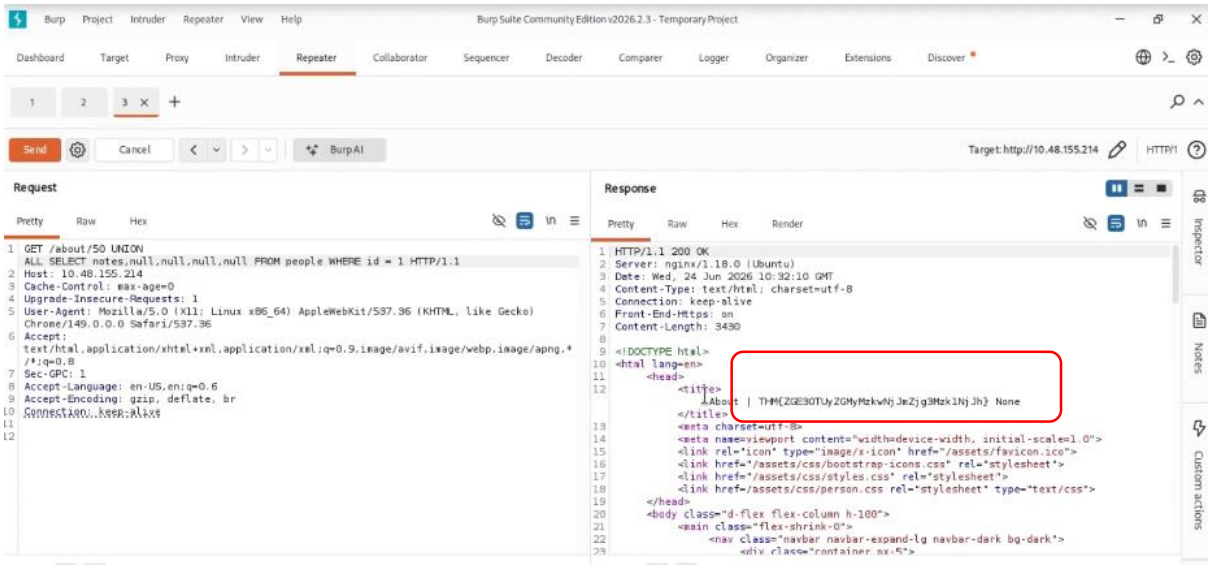
- **Fig. 3.1:** TryHackMe Burp Suite: Repeater room completion certificate and module dashboard.



- **Fig. 3.2:** Active manipulation of the Request Headers within Burp Repeater, successfully adding `FlagAuthorised: True` and extracting the `THM{Yzg2MWI2...}` flag from the `200 OK` response.



- **Fig. 3.3:** Endpoint validation testing in Repeater, replacing the product ID with `-1` (`GET /products/-1 HTTP/1.1`), successfully triggering a backend crash, receiving a `500 Internal Server Error`, and extracting the `THM{N2MzMz...}` flag.



- **Fig. 3.4:** Successful exploitation of the Extra-Mile SQL Injection Challenge. The screenshot displays the final payload `GET /about/0 UNION ALL SELECT notes,null,null,null,null FROM people WHERE id = 1 HTTP/1.1` crafted in Repeater, with the `THM{ZGE3OTUy...}` flag successfully extracted into the HTML `<title>` tag of the server response.

7. Results

The Burp Suite Repeater module was completed with a 100% success rate. Through the systematic use of the Repeater tool, I successfully intercepted standard web traffic, isolated it from the browser, and executed precise manual attacks. I successfully bypassed authorization checks via header injection, identified poor input sanitization by triggering `500 Internal Server Errors`, and executed a complex, multi-stage Union SQL Injection attack to extract protected database records.

8. Key Learnings

Completing this rigorous module yielded several critical insights into WAPT methodologies:

- **Repeater is the Core of WAPT:** While automated tools like Intruder or external scanners are useful for discovery, the actual exploitation and validation of complex vulnerabilities almost exclusively occur within the Repeater module. The ability to meticulously tweak a payload character-by-character and observe the exact byte-level response is irreplaceable.
- **Verbose Errors are Catastrophic:** The SQL Injection challenge highlighted the severe danger of improper error handling. By leaking the exact backend SQL query (`SELECT ... FROM people WHERE id = ...`) in the `500 Internal Server Error` response, the application handed the attacker the exact schema required to bypass the enumeration phase and immediately construct a functioning exploit.
- **Never Trust the Client:** The header manipulation task (`FlagAuthorised: True`) demonstrated that all data originating from the client—including HTTP headers, cookies, and URIs—must be treated as untrusted input. The server blindly trusted the injected header, resulting in a complete access control bypass.
- **SQLi Requires Patience and Precision:** Exploiting the Union SQLi required balancing column counts (`null,null,null,null`) and forcing the application to display the injected data by targeting an invalid row (`id=0`). Repeater's dual-pane view was critical for rapidly iterating these payloads until the syntax matched perfectly.

9. Conclusion

The TryHackMe Burp Suite Repeater room provided an exceptional, hands-on environment for mastering the most critical tool in a penetration tester's arsenal. Moving beyond the foundational interception capabilities of the Proxy module, Repeater proved to be a highly versatile, powerful workbench for dissecting application logic. By practically applying theoretical concepts to bypass authorization, trigger unhandled exceptions, and exploit a complex SQL Injection flaw entirely through manual request manipulation, a deep understanding of protocol-level exploitation was achieved. This mastery of Burp Suite Repeater establishes the necessary technical foundation for tackling automated fuzzing with Intruder and the advanced Web Vulnerability tasks (SQLi, XSS, BAC) required in Task 4 of the WAPT Phase 1 Internship.

10. References

1. TryHackMe - Burp Suite: Repeater: https://tryhackme.com/room/burpsuiterepeater
2. WAPT Phase 1 Task Instructions & Guidelines - IqraSec Academy